MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Agents in the Physical World

Wei-Chun Huang, Yun-De Liu, Abhi Patel

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

ASPIRE: Agentic /Skills Discovery for Robotics

Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Yuke Zhu, Linxi “Jim” Fan, Guanzhi Wang

Background

Traditional	VLA / Imitation	Code-as-Policy	ASPIRE
Hand-written robot control scripts	Behavior learned from demonstrations	LLM writes executable robot programs	Code, execution traces, repairs and transferable skills

From one-off task execution to cumulative debugging experience

Problems with Code-as-policy

1. Coarse feedback

“Task failed” does not say which perception, planning or grasp primitive broke.

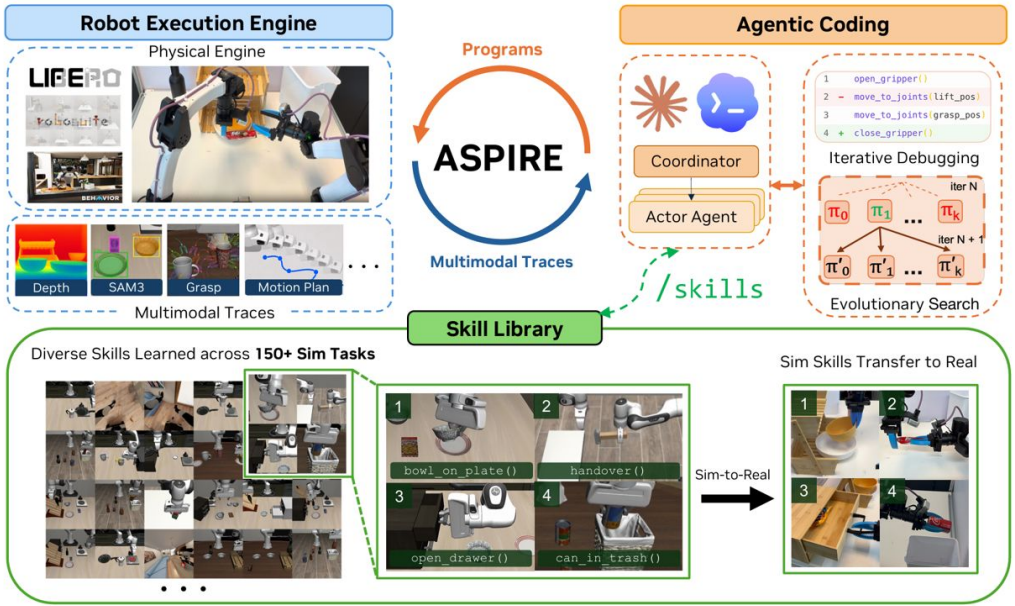
---

2. No durable learning

Successful fixes are discarded instead of reused on later tasks.

ASPIRE’s answer Fine-grained traces + persistent skills + diverse program search

The System



- Robot Execution Engine
  - Records program execution and multimodal, per-primitive traces.
- Skill Library
  - Stores validated repairs as reusable skills.
- Evolutionary Search
  - Generates diverse candidate programs and retains useful solutions.

Technical Details: Coordinator–Actor System

Coordinator

Dispatches tasks to parallel actors.

Audits successful repairs.

Serializes writes to the shared skill library.

Reruns low-scoring tasks with the final library.

Actor

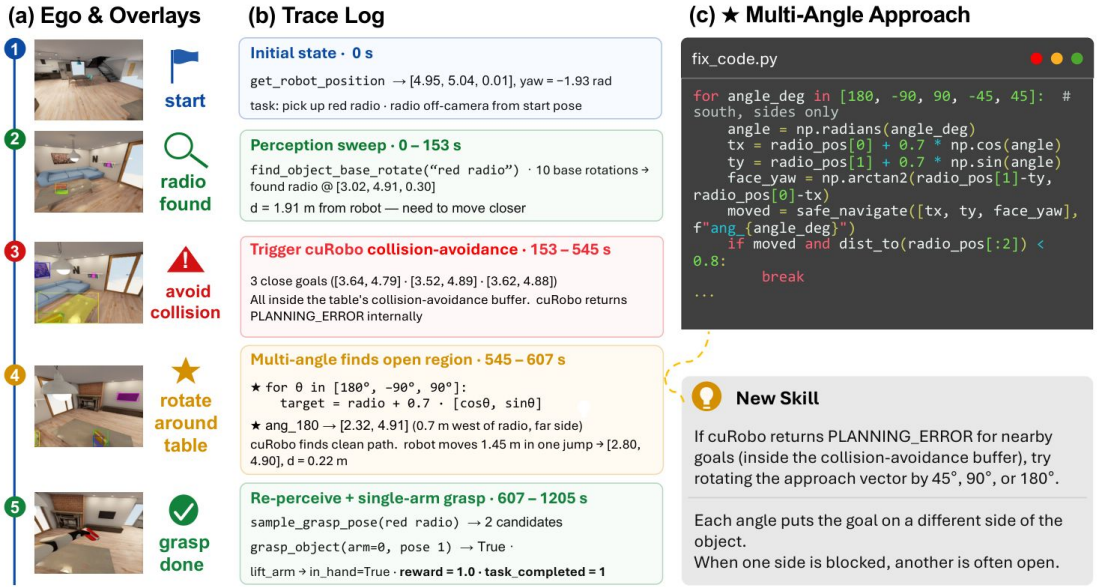
Writes code-as-policy programs.

Calls robot execution APIs.

Inspects multimodal traces and patches the failure.

Validates fixes by re-execution.

Robot Execution Engine



Trace-guided debugging: planning error → multi-angle repair → validated new skill

Technical Details: Skill Library

- **Heterogeneous Repair Distillation:** Stores validated, agent-discovered repairs as modular, transferable in-context guidance rather than fixed human-written code primitives.
- **Structured Entry Schema:** Captures the observed failure signature, when-to-apply activation guard, verified repair strategy, and representative code snippets.
- **Diverse Skill Categories:** Encompasses localization/disambiguation, perception prompting, grasping constraints, navigation recovery (e.g., multi-angle approaches), low-level motion primitives, scene reasoning, and debugging workflows.
- **Cross-Task & Cross-Embodiment Transfer:** Functions as in-context prompts for future actor agents to enable zero-shot transfer on unseen long-horizon tasks and cross-embodiment guidance for real-robot manipulation.

Technical Details: Evolutionary Search

1 Generate candidates

Propose K alternative task sequences and control programs, rather than patching one failed path.

---

2 Execute & select

Run candidates; inspect primitive traces; retain stronger programs and unresolved failure evidence.

---

3 Evolve & stop

Generate offspring from survivors, the skill library and a task-analysis ledger; stop on validated success or budget.

Evaluation Setup: Benchmarks, Baselines & Protocol

Three benchmarks evaluate different robotic capabilities

Benchmark	Evaluation focus
LIBERO-Pro	Short-horizon robustness: object, goal, spatial perturbations
Robosuite	Contact-rich single- and dual-arm manipulation
BEHAVIOR-1K	Long-horizon navigation and object manipulation

Coding Agent & Baselines

- ASPIRE: Claude Code + Opus 4.6 (1M context)
- Baselines: CaP-Agent0, OpenVLA, π0, π0.5, human-written programs

Evaluation Protocol

- Disjoint debug and held-out evaluation seeds
- LIBERO-Pro / Robosuite: one program per task
- BEHAVIOR-1K: incremental code from traces

Key Evaluation Distinction

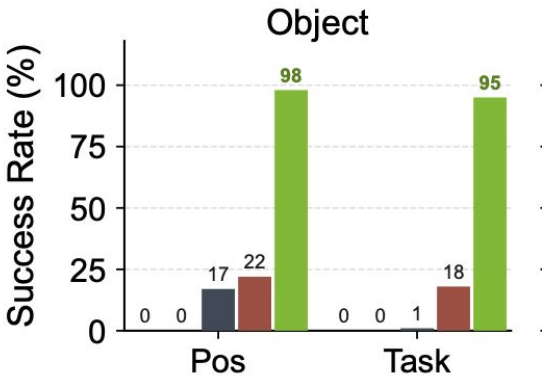
- ASPIRE => Reuses a program across unseen seeds
- CaP-Agent0 => Generates and retries programs separately for each seed (LIBERO-Pro / Robosuite)

Evaluation Focus: Robustness · Task Generalization · Skill Reuse · Real-World Transfer · Component Ablation

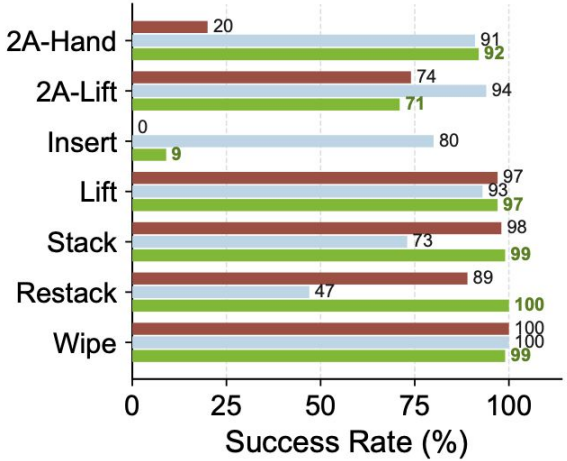
Overall results: Gains span robustness, contact and mobility



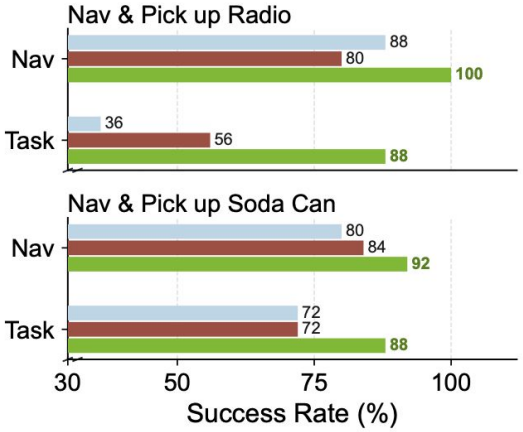
LIBERO-Pro: Object



Robosuite



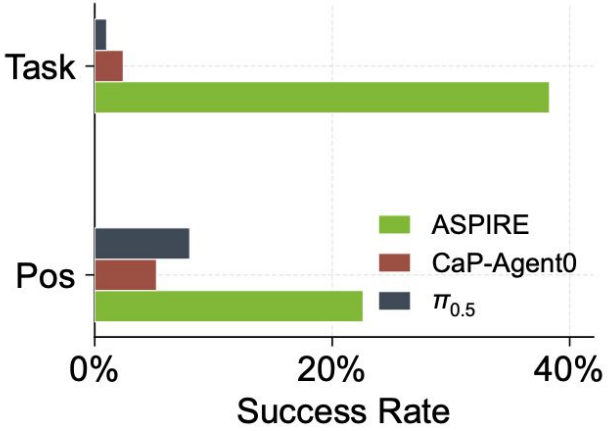
BEHAVIOR-1K



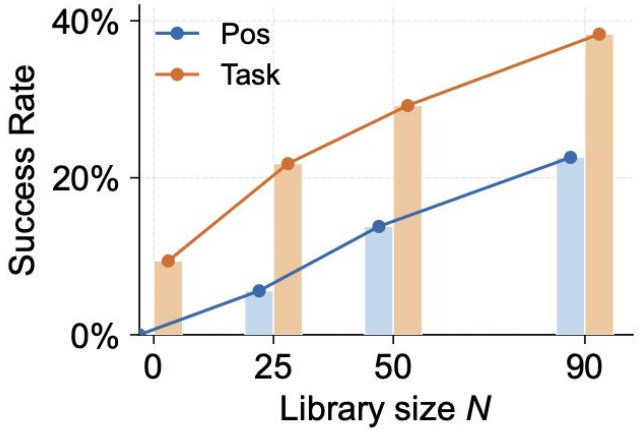
One code-as-policy harness improves results across three benchmark families

Skill reuse: Zero-Shot Skill Transfer to Unseen Long-Horizon Tasks

Skills accumulated from LIBERO-90 source tasks are reused on unseen LIBERO-Pro Long tasks, without target-task debugging or retries



(a) Full N=90 library zero-shot results versus baselines.



(b) Zero-shot success improves as skill-library size increases.

Skills learned from previous tasks improve zero-shot success with gains as source-task grows

Real-Robot Transfer: Can Simulation Skills Generalize Across Embodiments?

Franka simulation → text-based skill guidance → real bimanual YAM robot
Codex + GPT-5.5 (xhigh); real-robot debugging with vs. without skill retrieval

Task	Output Tokens (M) ↓		Total Tokens (M) ↓		Success Rate ↑	
	w/o Skills	w/ Skills	w/o Skills	w/ Skills	w/o Skills	w/ Skills
Put bowl on plate	0.05	0.04	8.65	5.11	20/20	20/20
Lift soda can	0.18	0.03	61.94	6.58	13/20	19/20
Open/push drawer	1.33	0.36	334.92	81.67	0/20	11/20

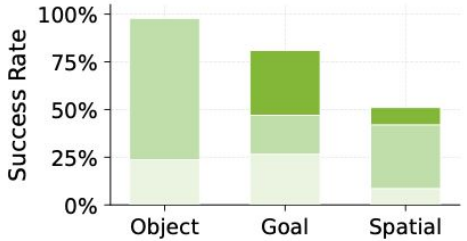
Skill retrieval reduces debugging cost and improves difficult tasks

- Soda-can lifting: 65% → 95% success; 89% fewer total tokens
- Drawer manipulation: 0% → 55% success; 76% fewer total tokens

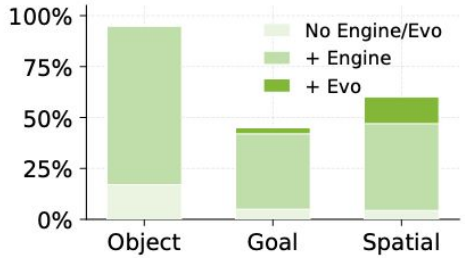
ASPIRE transfers reusable debugging knowledge, not executable policies

Ablation: actionable traces drive gains; search adds reliability

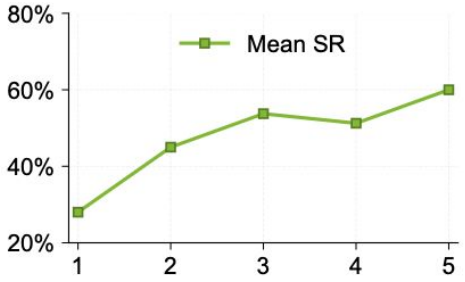
LIBERO-Pro: isolate the execution engine, then add evolutionary search



(a) Position perturbation.



(b) Task perturbation.



(c) Per-iteration progress.

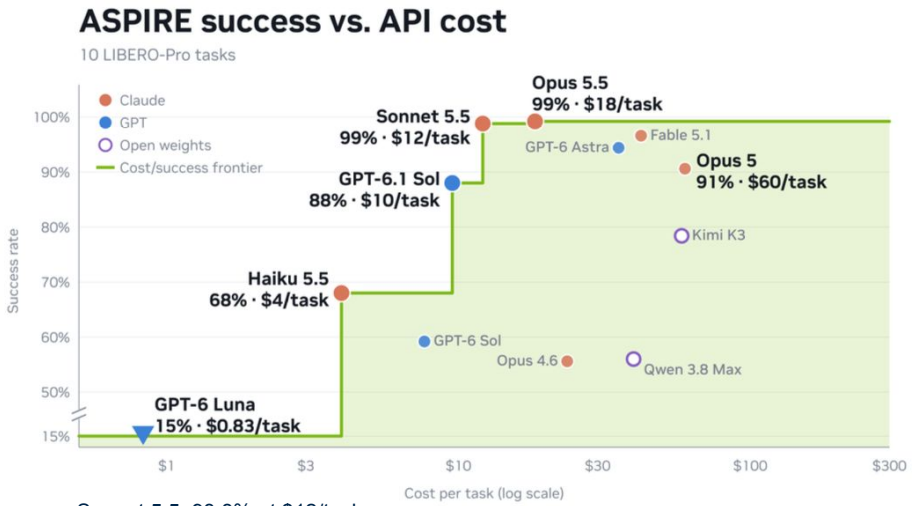
14% → 62% → 72%

No engine/search → + engine → + search

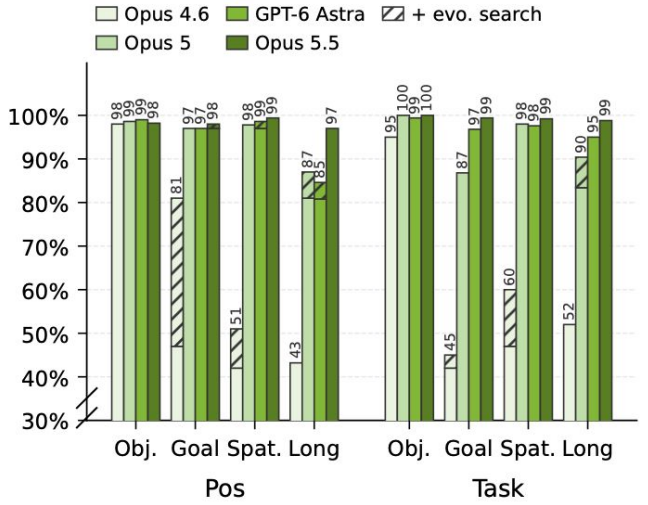
Dense traces provide most of the gain.
Search helps residual hard cases;
extra rounds have uneven, smaller gains.

Actionable feedback and exploration play complementary roles

Model ablation: Stronger LLMs Improve Success and Cost Efficiency



Sonnet 5.5: 98.8% at $12/task
Opus 5.5: 99.2% at $18/task
Opus 5: 91% at $60/task



Object / Goal / Spatial: 96–99% with stronger models
Evolutionary search: smaller average gains, but useful for hard tasks

Stronger LLMs improve program synthesis, while evolutionary search remains valuable for hard tasks

Case study: stronger models change the manipulation strategy

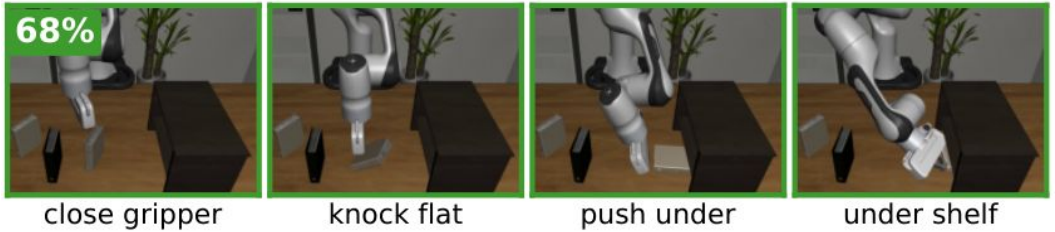
Task: place the book under a shelf whose gap is too low for an upright grasp.

Opus 4.6
Grasp and carry
0% success



Book stays above the target shelf.

Opus 5.5
Knock flat + push
68% success



Changes strategy: flattens the book, then pushes it under.

Reconsider the manipulation strategy when the default grasp-and-move plan fails

Strengths & Limitations

Strengths	Limitations
Execution traces enable targeted debugging	Debugging and search require many rollouts
Reusable skills improve cross-task generalization	Performance depends heavily on the base LLM
Simulation skills transfer across robot embodiments	Retrieved skills may become outdated or irrelevant
Execution feedback validates generated programs	Small real-robot study, with human resets

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

TimelyLLM: Time-sensitive LLM Serving System for Physical-I/O Limited Agents

Neiwen Ling, Guojun Chen, Anurag Khandelwal, Lin Zhong

What are Physical-I/O limited agents?

Unlike text-based agents, they consume LLM outputs sequentially at a **rate constrained by physical execution or speech playback**

Robotic agents: act in the world



(a) Drone (b) Robot Arm (c) Quadruped Robot

Ryze Tello drone: move / turn
Neuromeka Indy7 Pro arm: pick / place
Unitree Go2 Pro: walk / navigate

Spoken agents: talk to users

Prototype evaluated in the paper

Whisper (speech recognition)
→ TimelyLLM (response generation)
→ PiperVoice (speech synthesis)

Responses must be played in order at a natural speech rate; extra generated text cannot be spoken instantaneously.

Motivation: Physical Execution Have Different Timing Constraints

LLM service latency (Table 1)

Phase	Tokens	Llama3	Phi3
Tokenization	2884	2.609 ms (0.43%)	1.541 ms (0.18%)
Decoding 0	token 0	328.45 ms	563.04 ms
Decoding 1	token 1	21.93 ms	22.24 ms
Decoding 2	token 2	21.56 ms	20.99 ms
3 to N-1	...	...	...
Decoding N	last token	21.77 ms	22.00 ms
Decoding	15	611.2 ms (99.57%)	841.7 ms (99.80%)
Detokenization	15	0.052 ms (0.008%)	0.091 ms (0.011%)
Total time	in:2884, out:15	613.82 ms	843.36 ms

Agent consumption rates (Table 2)

Robot Type	Forward	Backward	Left	Right	Turn
Drone(Ryze Tello)	2.53s	2.53s	2.53s	2.53s	1.53s
Quadruped (Unitree Go2)	1.50s	1.50s	2.47s	2.47s	0.80s
Self-deployed Robot Car	3.87s	3.87s	4.08s	4.08s	1.17s

(a) Robot-side action execution time ($d = 1$ m, $\theta = 45°$).

	ChatGPT	Gemini	Meta AI	Siri
Time(20 words)	8.43s	7.7s	8.14s	7.58s
Avg Words Per Minute	142	156	147	158

(b) Speech playback speed of major spoken agents.

Generation: >99% of measured time

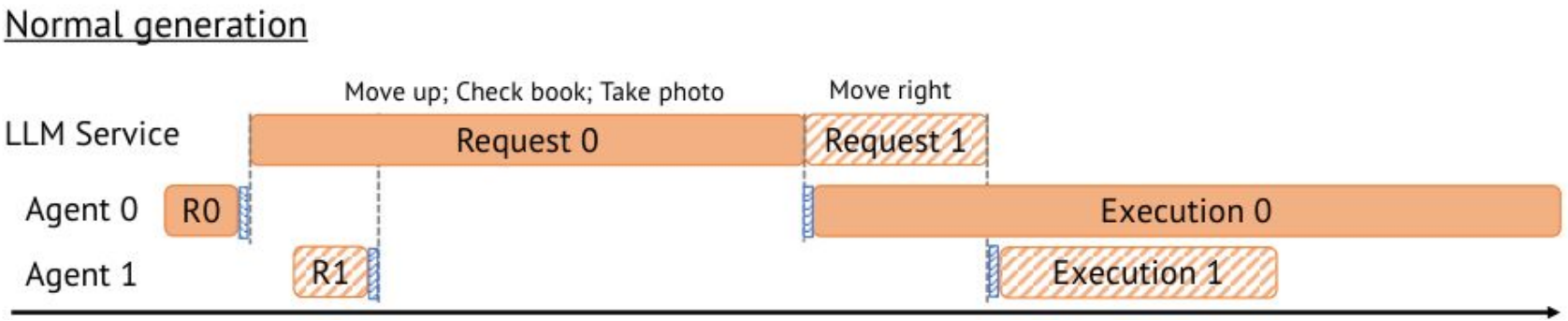
The first token includes input prefill.
Later tokens benefit from KV-cache reuse.

Robots: seconds per motion
Speech: 142–158 words/min

Physical execution creates opportunities to overlap LLM generation, but scheduling must prevent agents from waiting

Why conventional serving delays urgent work

R0: inspect a book (move up → check book → take photo). R1: avoid a collision.



1 Too-early generation

Future steps use compute before the agent needs them.

2 No execution-aware yield

The full plan occupies service; physical slack is not exploited.

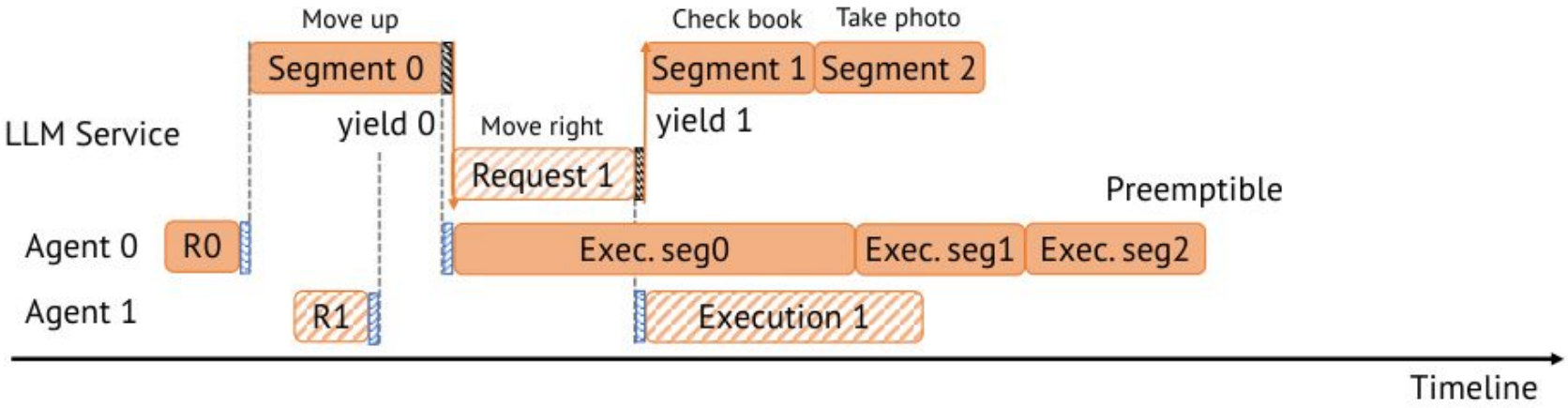
3 Urgency-blind ordering

Arrival order can delay urgent collision avoidance.

Useful tokens must also arrive at useful times

Generate one executable segment, then yield and resume

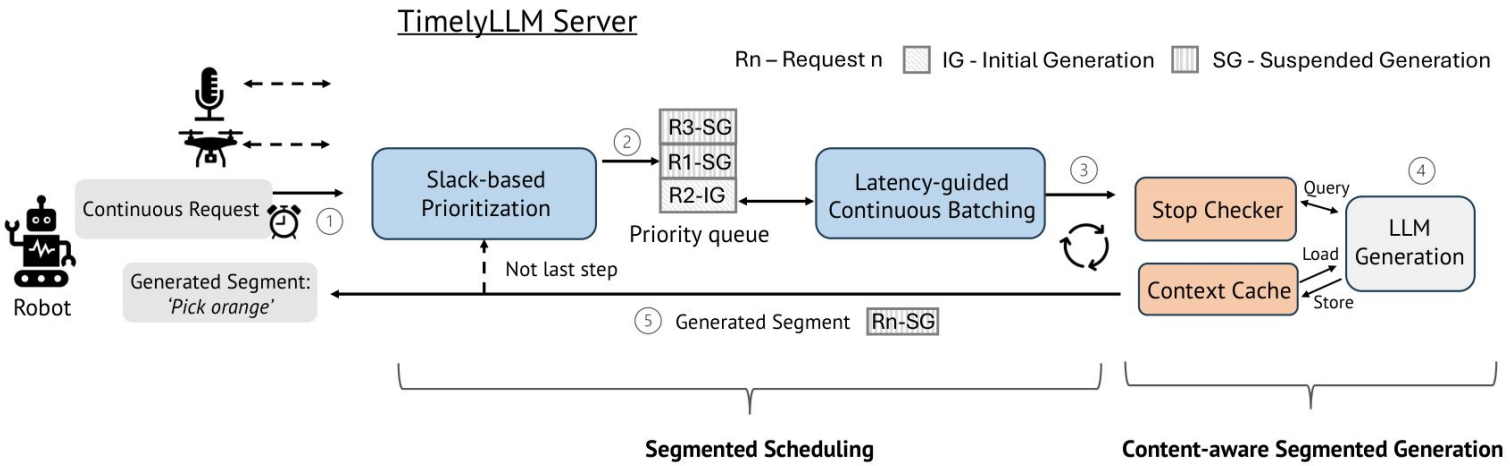
R0: book inspection (solid orange). R1: collision avoidance (hatched orange).



Yield while Agent 0 executes; resume before its next segment is needed.

Response time: request arrival → agent starts acting or speaking.
Completion time: total LLM waiting + physical execution / speech playback.

TimelyLLM system overview



Segment

Emit an executable unit;
yield and preserve its context

Prioritize

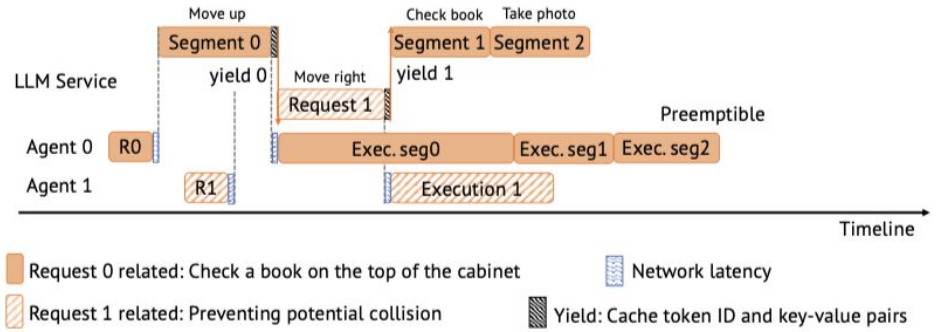
Order initial and suspended requests by slack and urgency

Batch

Expand the batch only if urgent work remains feasible

Content-Aware Segmentation

Split the output into meaningful, executable segments.



A segment is ready only when it passes two checks

- **Structural Completeness**
- **Time Compliance**

$$W(s_k^i) = max\{0, GS(s_k^i) + G + C - EE(s_{k-1}^i)\}$$

GS: generation start G: generation time C: transmission EE: previous action end

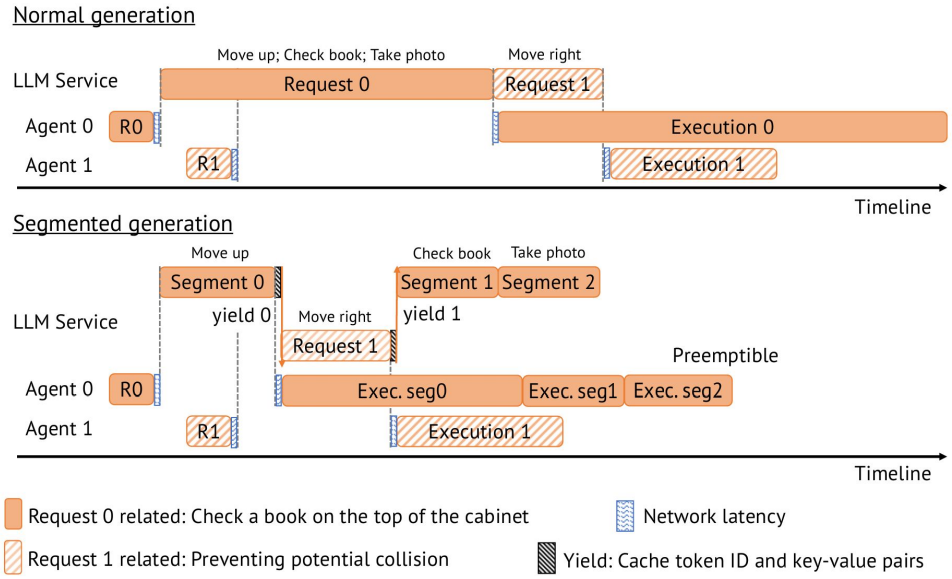
Two Things go into the context cache:

KV Cache

Output Token IDs

Goal: maximize total time utility, balancing first response and later waits

Slack-Based Scheduling



Δ < 0: time to spare

Generation can wait without stalling the agent

Δ = 0: just in time

Next segment arrives as the current action ends

Δ > 0: lateness risk

Next segment is predicted to arrive after it is needed

$$\Delta(s_k^i, t) = t + G + C - EE(s_{k-1}^i)$$

$$Pri(s_k^i, t) = \left(\alpha^i \max(0, \Delta(s_k^i, t)) + \beta^i\right) \cdot e^{\min(0, \Delta(s_k^i, t))}$$

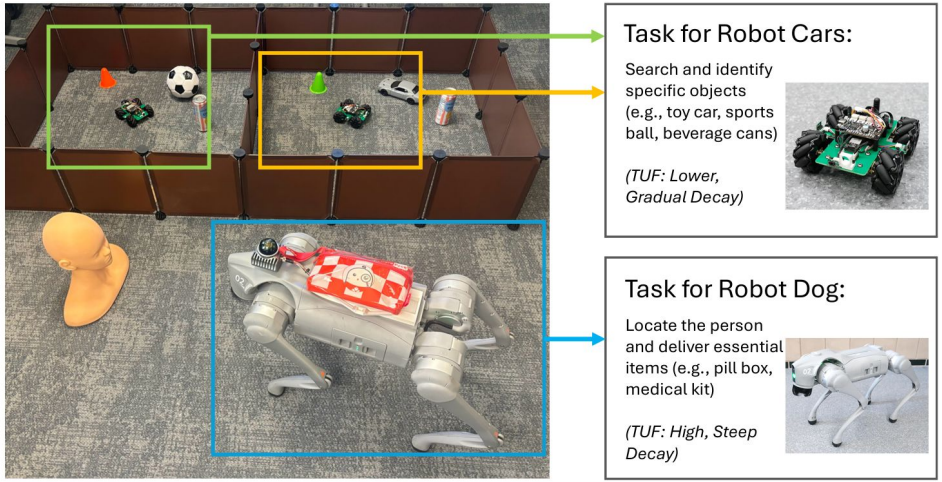
Evaluations

- Methods
  - Main Benchmark: LRTrace
  - Real World Evaluations
    - Multi-Robot Testbed
- Setup:
  - Models: Llama3-8B-float16.
  - Hardware:
    - NVIDIA RTX 4090 GPU
    - AMD Ryzen 9 7950X CPU
    - 24 GB of host memory.

Categories	Trace ID	Task Description
Normal request for drone	1	Look for the cat
	2	Check for keys or wallets
	3	Check the book on the cabinet
	4	Find a toy under the bed
	5	Find a tool for cutting paper
Urgent request for drone	6	Avoiding human injury
	7	Preventing potential collision
	8	Evading animal pursuit
Complex request for robot arm	9	Object stacking
	10	Desk cleaning
	11	Fruit and vegetable classification
Request for quadruped robot	12	Find the household item
	13	Guide to the door
	14	Bark to deter intruders

WID	Categories	EPS	Max TPE	Trace ID	AN	Time (s)
1	Mixed TUF	0.25	8	1-8	25	260
2	High-Parallel	0.25	16	1-8	42	300
3	Arm - *TypeFly*	0.1	8	9-11	40	900
4	Arm - *FLTRNN*	0.5	8	9-11	81	100
5	Mixed Robots	0.25	12	3-8,12+	47	500

Real World Evaluation: Robot Agents



Task for Robot Cars:

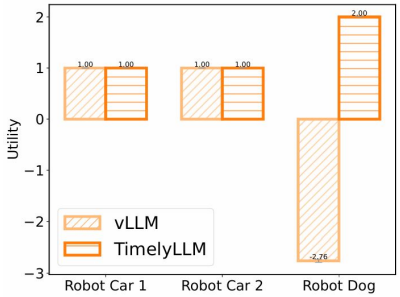
Search and identify specific objects (e.g., toy car, sports ball, beverage cans)

(TUF: Lower, Gradual Decay)

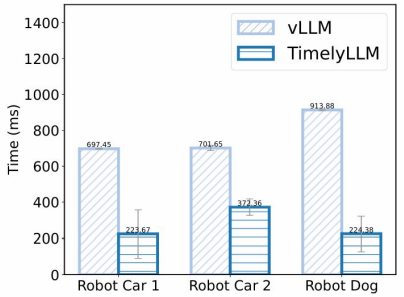
Task for Robot Dog:

Locate the person and deliver essential items (e.g., pill box, medical kit)

(TUF: High, Steep Decay)



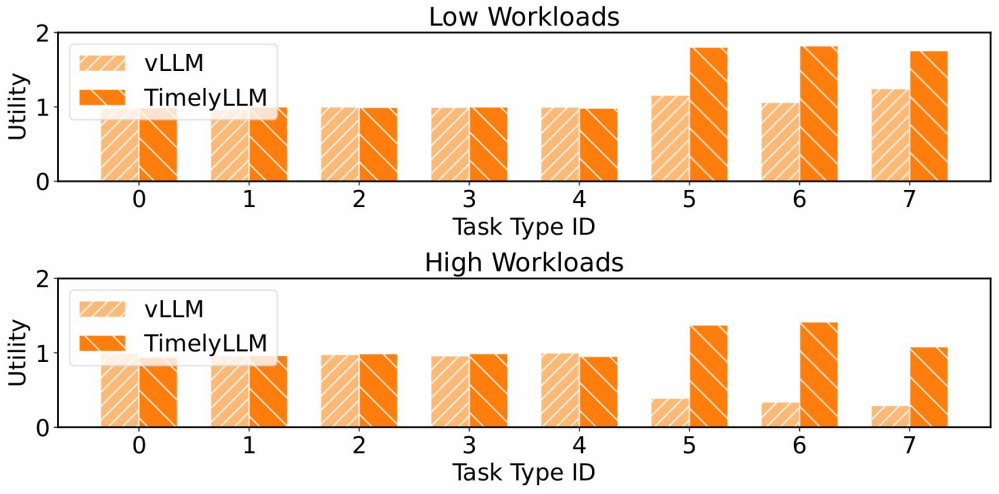
(a) Time Utility



(b) Waiting Time

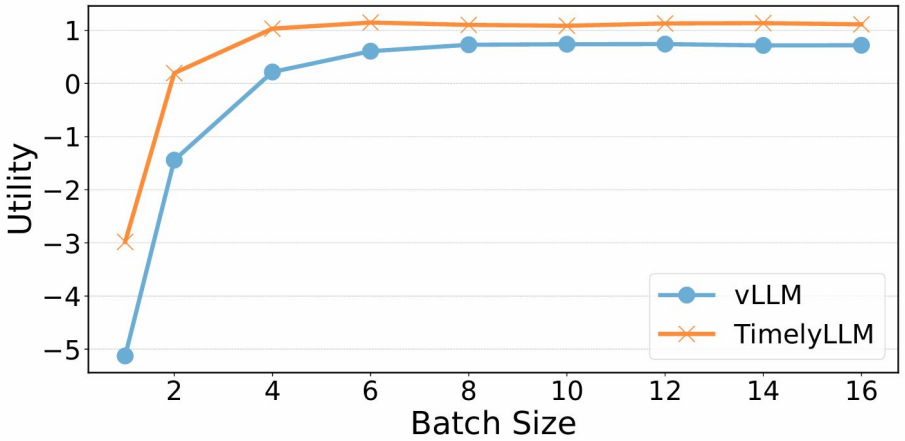
Robot car: ERT = 1 s, beta = 1, alpha = -2
Robot dog: ERT = 200 ms, beta = 2, alpha = -6.67

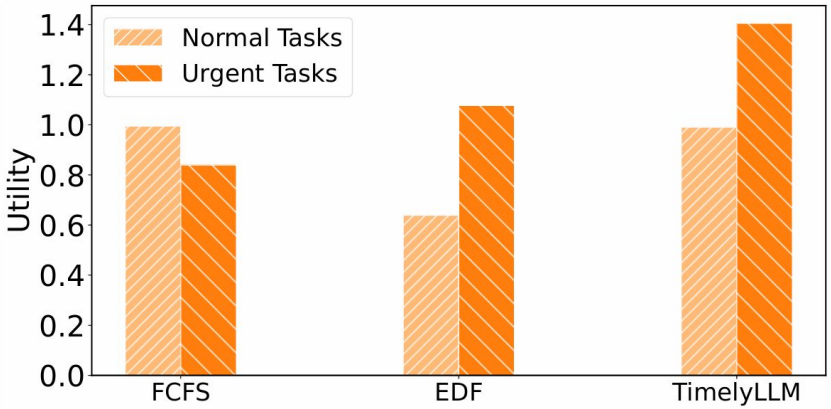
How does Timely LLM perform in time sensitivity?



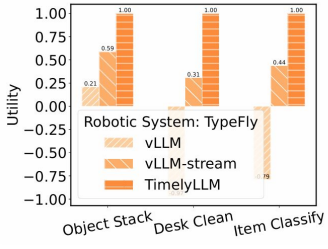
WID	Categories	EPS	Max TPE	Trace ID	AN	Time (s)
1	Mixed TUF	0.25	8	1-8	25	260
2	High-Parallel	0.25	16	1-8	42	300
3	Arm - *TypeFly*	0.1	8	9-11	40	900
4	Arm - *FLTRNN*	0.5	8	9-11	81	100
5	Mixed Robots	0.25	12	3-8,12+	47	500

Which system components enhances TimelyLLM?

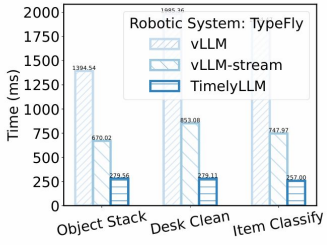




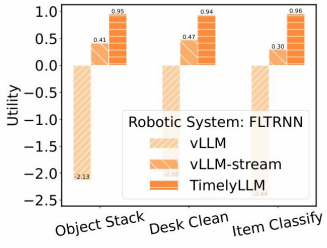
Does TimelyLLM Generalize?



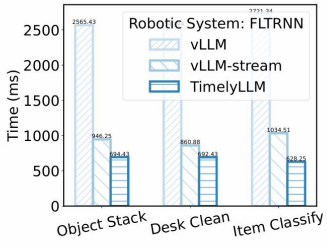
(a) Time Utility-TypeFly



(b) Waiting time-TypeFly



(c) Time Utility-FLTRNN



(d) Waiting time-FLTRNN

WID	Categories	EPS	Max TPE	Trace ID	AN	Time (s)
1	Mixed TUF	0.25	8	1-8	25	260
2	High-Parallel	0.25	16	1-8	42	300
3	Arm - *TypeFly*	0.1	8	9-11	40	900
4	Arm - *FLTRNN*	0.5	8	9-11	81	100
5	Mixed Robots	0.25	12	3-8,12+	47	500

System Overhead

Table 6: *The segmented overhead across eight traces, presented in milliseconds and as a percentage.*

Trace ID	1	2	3	4	5	6	7	8
Overhead (ms)	13.7	24.1	19.9	19.9	16.0	13.1	14.4	14.8
Overhead (%)	3.0	2.6	5.6	2.8	3.2	3.7	4.1	4.2

Table 7: *The scheduling overhead for various sizes of task queues.*

Task Number	1	2	4	6	8
Scheduling Overhead (ms)	0.58	0.83	1.25	1.58	2.03

Strengths & Limitations

Strengths

Clear insight. Robots act slower than LLMs generate, so execution time is slack worth reclaiming.

Low overhead. Segmentation under 6% of generation time; scheduling about 2 ms.

Real systems. Physical robots and a voice pipeline, built on vLLM, with code and dataset released.

Theory plus ablations. Competitive-ratio bound, and FCFS/EDF tests isolate the scheduler.

Limitations

Needs slack. Smaller gains for tight feedback loops like ReAct, as the authors note.

Hand-set parameters. Priorities depend on chosen α, β and profiled action times.

Small scale. One RTX 4090, 7–8B models, up to 16 tasks; multi-GPU only discussed.

Narrow comparison. Self-built benchmark; baselines are mostly vLLM variants.



Q & A